COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

# Cloud Specialist [ D68444-15-2025-23 ]

## Unità Formativa : Programmazione Python

## Docente: Massimo PAPA

## Titolo argomento: Soluzione esercizi sulle tuple

Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

# Esercizi sulle tuple


**Indicazioni sulla consegna** **3**
Struttura del codice sorgente 4
**Esercizi sulle tuple** **6**
Primo Esercizio 6
Secondo Esercizio 6
Terzo Esercizio 6
Quarto Esercizio 6
**Soluzione esercizi sulle tuple** **7**
Primo Esercizio 7
Secondo Esercizio 9
Terzo Esercizio 10
Terzo esercizio BIS 12
Prima soluzione: 12
Seconda soluzione: 13
Quarto Esercizio 14

# Indicazioni sulla consegna

Implementare gli algoritmi risolutivi dei seguenti esercizi codificandoli in linguaggio python. Partire analizzando il problema seguendo la seguente traccia:

- Quali sono gli input del problema?
- Quali sono gli output?
- Suddividi il problema in problemi più semplici e se lo ritieni opportuno descrivi la soluzione di ogni sottoproblema con un flow-chart
- Andrai a rappresentare ogni algoritmo che risolve un sottoproblema come una funzione
- La funzione restituisce un valore? Se si di che tipo?
- Ogni funzione accetta una lista di parametri formali? Se si di che tipo?
- Esegui la codifica partendo dal programma principale, al suo interno vai a richiamare le funzioni che poi andrai successivamente a definire.
- Continua la codifica dichiarando e definendo tutte le funzioni prima del programma principale che richiama tutte le altre. In testa alle funzioni scrivi un commento che descrive la funzione stessa , cosa restituisce e quali sono i parametri formali.
- Scrivi la sezione dell'inizializzazione variabili scrivendo in un commento il significa della variabile stessa
- Verifica la codifica utilizzando input di test, cercando di provare anche i casi limite.

Attenzione: non utilizzare strutture complesse oltre alle tuple

## Struttura del codice sorgente

Relativamente alle indicazioni di scrittura del codice, utilizza il seguente schema generale:

```
'''
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Testo esercizio

'''

##
## Funzioni:
##


def fun(param1: int, param2: float) -> int:
  '''
     Funzione: fun
      Template per costruire le funzioni
      Parametri formali:
          int Param1 -> descrizione Param1
          float Param2 -> descrizione Param2
      Valore di ritorno:
          int -> descrizione valore di ritorno
  '''
  retValue = 5 # Valore di ritorno della funzione

  return retValue


'''
    Programma principale
    Descrizione sintetica funzionalità
    del programma principale.
'''

# Sezione di input Dati

# Inizializzazioni variabili

# Elaborazione
```

```
# Eventuali sotto processi di Elaborazione

# Sezione di output
```

# Esercizi sulle tuple

## Primo Esercizio

Scrivere un programma per rimuovere l'n- esimo carattere da una tupla non vuota.

## Secondo Esercizio

Scrivere un programma per invertire una tupla

```
Esempio:
     tpleIN=('a','c',f')
     tpleOUT=('f','c','a')
```

## Terzo Esercizio

Scrivere un programma per sostituire l'ultimo valore delle liste in una tuple con un valore richiesto in input.

```
Esempio:
     valore : 100
     TuplaIN: ([10, 20, 40], [40, 50, 60], [70, 80, 90])
     TuplaOUT: ([10, 20, 100], [40, 50, 100], [70, 80, 100])
```

## Quarto Esercizio

Scrivere un programma per contare gli elementi in una lista finché non si incontra un elemento di tipo tupla. [Suggerimento: si usi la funzione isinstance( )]

# Soluzione esercizi sulle tuple

## Primo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Primo Esercizio
    Scrivere un programma per rimuovere l'n- esimo
    carattere da una tupla non vuota.

"""


def rimuoviCarattere(stringa: str, n: int) -> str:
    """
    Funzione: rimuoviCarattere
    Permette di rimuovere l'ennesimo carattere dalla
    stringa in input alla funzione
    Parametri formali:
        str -> stringa da modificare
        n -> posizione carattere in stringa da rimuovere
    Valore di ritorno:
        str -> stringa modificata

    **************************************************
    * ATTENZIONE:                                    *
    *  FUNZIONE COPIATA DA ESRCIZIO SULLE STRINGHE N.1 *
    **************************************************

    """
    # porzione stringa che precede carattere n
    primaParte = stringa[:n]  # porzione stringa che precede carattere n
    # porzione stringa che succede a carattere n
    if n == -1:  # gestisco il caso particolare
        secondaParte = stringa[-1:0]  # sequenza vuota
    else:
        secondaParte = stringa[n + 1 :]
    return primaParte + secondaParte


"""
```

```
Programma principale
Si prova la funzione di rimozione elemento utilizzando
la funzione di rimozione carattere per le stringhe.
"""

# Sezione di input (input hard-coded)
tupla = ("a", 3, [2, "b"])
pos = -1

# Elaborazione
tupla = rimuoviCarattere(tupla, pos)

# Sezione di output
print(tupla)
```

## Secondo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Secondo Esercizio
    Scrivere un programma per invertire una tupla
    Esempio:
        tpleIN=('a','c',f')
        tpleOUT=('f','c','a')

"""


def invertiTupla(tupla: tuple) -> tuple:
    """
    Funzione: invertiTupla
    Inverte gli elementi di una tupla
    Parametri formali:
        tuple -> tupla da invertire
    Valore di ritorno:
        tuple -> tupla invertita

    """
    return tupla[::-1]


"""
Programma principale
Si prova la funzione di inversione elementi
della tupla
"""

# Sezione di input (input hard-coded)
tuplaIN = ("a", 3, [2, "b"])
tuplaIN = ()

# inversione elementi
tuplaOUT = invertiTupla(tuplaIN)

# Sezione di output
print(tuplaOUT)
```

## Terzo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Terzo Esercizio
    Scrivere un programma per sostituire l'ultimo valore
    delle liste in una tuple con un valore richiesto
    in input.
    Esempio:
        valore : 100
        tuplaIN: ([10, 20, 40], [40, 50, 60], [70, 80, 90])
        tuplaOUT: ([10, 20, 100], [40, 50, 100], [70, 80, 100])

"""

def hasLista(tupla: tuple) -> bool:
    """
    Funzione: verificaTupla
    verifica che la tupla abbia come elementi
    solo liste.
    Parametri formali:
        tuple tupla-> tupla da verificare
    Valore di ritorno:
        bool -> errore se False, altrimenti True

    """

    # Inizializzo valore di ritorno
    hasLista =  True

    # loop di verifica
    for ele in tupla:
        if not isinstance(ele,list): # se almeno un ele no lista
            hasLista =  False        # esco dal loop con errore
            break
    return hasLista

def sostituisciElementoLista(tupla: tuple,ele:any) -> tuple[tuple,bool]:
    """
    Funzione: sostituisciElementoLista
    Sostituisce ele all'ultimo elemento della lista
    contenuta nella tupla di input.
```

```
    Se elementi della tupla non sono liste allora
    si rileva un errore
    Parametri formali:
        tuple tupla-> tupla da modificare
        any ele -> elemento da sostituire nella lista
    Valore di ritorno:
        tuple -> tupla modificata
        bool -> errore se False, altrimenti True

    """

    # inizializzo tupla di output
    tuplaOUT=()
    # Verifico che la tupla sia corretta
    noErrore = hasLista(tupla)
    if noErrore:
        for i in range(len(tupla)):
            # Verifico che la tupla non sia vuota
            if len(tupla[i]) > 0:
                tupla[i][-1]=ele
            else:
                tupla[i].append(ele)

    return noErrore,tupla


"""
Programma principale
Si prova la funzione di modifica elementi
lista nella tupla
"""

# Sezione di input (input hard-coded)
tuplaIN = ([10, 20, 40 ], [40, 50, 60 ], [70, 80, 90 ])
tuplaIN = ([10, 20, 40 ], [], [70, 80, 90 ])
valore = 'a'

# Modifica lista
noErr,tuplaIN=sostituisciElementoLista(tuplaIN,valore)

# Sezione di output
if noErr:
    print(tuplaIN)
else:
    print("ERR: tupla non correttamente formata!!")
```

## Terzo esercizio BIS

Scrivere un programma per **sostituire** l'ultimo **valore** delle **tuple in una lista** con un valore richiesto in input.

Esempio:

valore : 100

TuplaIN: [**(10, 20, 40)**, (40, 50, 60), (70, 80, 90)]

TuplaOUT: [**(10, 20, 100)**, (40, 50, 100 ), (70, 80, 100)]

### Prima soluzione:

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Terzo Esercizio BIS
    Scrivere un programma per sostituire l'ultimo valore delle tuple
    in una lista con un valore richiesto in input.
    Esempio:
        valore : 100
        TuplaIN: [(10, 20, 40), (40, 50, 60), (70, 80, 90)]
        TuplaOUT: [(10, 20, 100), (40, 50, 100 ), (70, 80, 100)]

"""

# Sezione di input (input hard-coded)
tuplaIN= [(10, 20, 40), (40, 50, 60), (70, 80, 90)]
valore = 100

# Elaborazione principale
tuplaOUT = []
for tupla in tuplaIN:
   tuplaModificata=tupla[:-1]+(valore,)
   tuplaOUT.append(tuplaModificata)

# Sezione di output
print(tuplaIN)
print(tuplaOUT)
```

### Seconda soluzione:

```
# Altra soluzione con le list comprehension
tuplaOUT = [tupla[:-1] + (valore,) for tupla in tuplaIN]
print(tuplaOUT)
```

## Quarto Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Quarto Esercizio
    Scrivere un programma per contare gli elementi in una
    lista finché non si incontra un elemento di tipo tupla.
    [Suggerimento: si usi la funzione isinstance( )]

"""

def contaFinoATupla(lista: list) -> int:
    """
    Funzione: contaFinoATupla
    Conta gli elementi di una lista passata in input
    sino a che incontri un elemento di tipo tupla.
    Parametri formali:
        list lista-> lista di input
    Valore di ritorno:
        int -> numero di elementi prima di una tupla

    """

    # Inizializzo contatore, valore di ritorno
    contatore =  0

    # loop di conteggio elementi
    for ele in lista:
        if isinstance(ele,tuple):
            break
        contatore += 1

    return contatore

"""
Programma principale
Si prova la funzione di conteggio elementi
nella lista
"""

# Sezione di input (input hard-coded)
lista = ['a', 1, True, 'b', 5.4, (40, 50, 60), 'i3']
```

```
lista = ['a', 1, True, 'b', 5.4,  60, 'i3']
lista = [(),3,4]

# Conto elementi
numeroElementi=contaFinoATupla(lista)

# Sezione di output
print("Numero di elementi prima di tupla: ",numeroElementi)
```